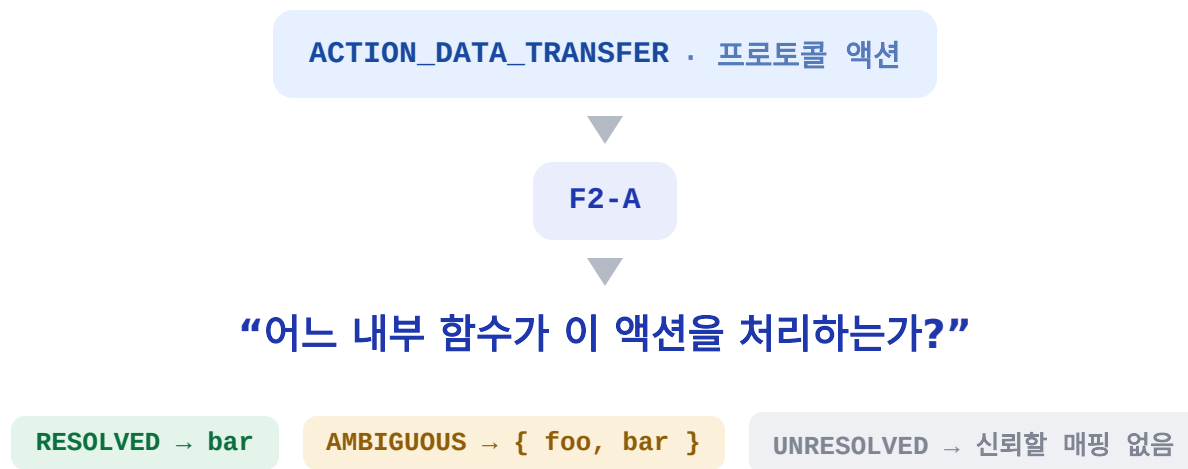


F2-A는 하나의 질문에 답한다



이 질문이 왜 중요한가

보안 질문 — “chargingLimit 은 검증되는가?”



필요로 함

핸들러 진입 함수



가능케 함

필드 바인딩

오염 소스

데이터 흐름

싱크·검사

핸들러가 없으면 하위 프로토콜 인지 분석에 **신뢰할 출발점이 없다**

입력과 출력 — 인터페이스부터

입력 1 · KB 액션 정의

canonical name · symbols · numeric IDs · aliases

입력 2 · 프로그램 모델

AST / CPG · 호출 · 대입 · 초기화 · 심볼 해석



출력 · 액션당

status · **chosen**(확정 시) · ranked candidates · evidence · conflict · unresolved reason

아직 아키텍처가 아니라 **인터페이스**를 정의한다

F2-A가 발견해야 하는 것 — 소스 구조에서 의미 관계

```
static Entry table[] = {  
    { ACTION_DATA_TRANSFER, foo }  
};  
void dispatch(Frame *f) {  
    table[f->action].fn(f->payload);  
}
```



F2-A가 발견

action	ACTION_DATA_TRANSFER
callback	foo
relation	같은 등록 항목에 속함
meaning	action → handler 후보

F2-A는 소스 구조로부터 **의미 관계(action→handler)**를 재구성한다

F2-A가 하지 않는 것

F2-A 한다

- 액션 → 핸들러 해석
- 후보 수집
- 증거 보존
- 신뢰도 · 순위
- 모호 · 실패 보고

F2-A 안 한다

- 페이로드 필드 → 변수 추적
- 오염 경로 계산
- 싱크 분류
- 프로토콜 검사 검증
- 최종 취약점 판정

F2-A 출력



하위 F3 · F4 · F5 · F6 분석

전체 알고리즘 — 여섯 문장

- 1 **프로토콜** **액션** **하나**를 잡는다.
- 2 그 액션을 **언급·인코딩하는 코드** 위치를 찾는다.
- 3 그 위치들의 **콜백을 모두** 뽑는다.
- 4 관찰을 **증거**로 바꾼다.
- 5 콜백별로 증거를 모아 **후보**를 **비교**한다.
- 6 **확정**하거나, **모호**를 남기거나, **실패 사유**를 보고한다.

사전확률·noisy-OR·provenance는 아직 언급하지 않는다 — **먼저 알고리즘**, 점수 정책은 나중

한 액션에 후보가 여럿일 수 있다

등록

```
{ ACTION_DATA_TRANSFER, foo }
```

✗ “첫 콜백을 찾는다”가 아니다

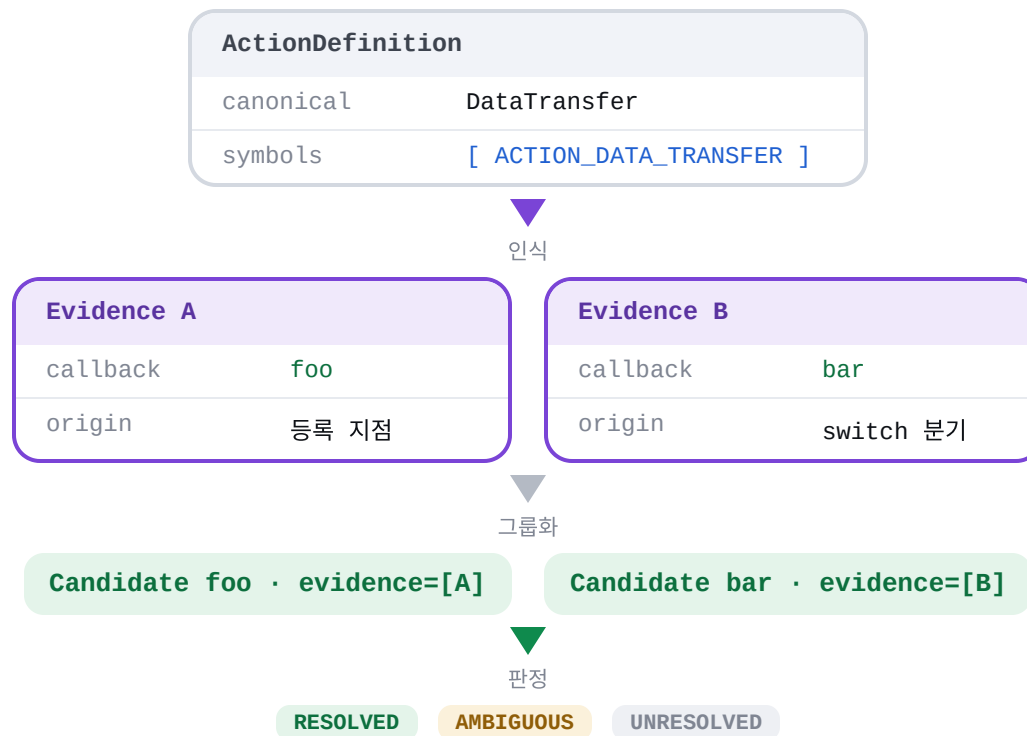
SWITCH

```
case ACTION_DATA_TRANSFER:
    bar(f->payload);
```

✓ **foo·bar**를 모두 모으고, 출처를 보존하고, 비교하고, 충분히 신뢰되는지 판단한다

그래서 **후보 해석(candidate resolution)**이라는 문제가 생긴다

F2-A가 실제로 만드는 객체들



증거가 필요한 이유

증거 없이

등록 추출기 → foo

switch 추출기 → bar

공통 비교 모델이 없는 충돌

증거로

둘 다 같은 스키마

출처·방식을 보존

하나의 계산이 비교

하나의 판정 계층이 결정

증거는 인식과 판정 사이의 공통 언어다 — 충돌하는 관찰을 풀기 위한 필수 장치

F2-A가 성공·주저·중단하는 때

RESOLVED

강한 후보 하나 · 또는 나머지보다 뚜렷이 강한 하나

AMBIGUOUS

그럴듯한 후보 둘 · 어느 것도 뚜렷이 우세하지 않음

UNRESOLVED

후보 없음 · 약한 증거뿐 · 미지원 구조 · 외부/누락 정의

수치 규칙 이전에 **동작 계약**부터 — 언제 확정하고, 언제 멈추는가

집합 초기화 — 같은 항목의 두 자식을 한 쌍으로

{ ACTION_DATA_TRANSFER, foo }

▼
AST로 관찰

```
initializer
├─ element[0]: ACTION_DATA_TRANSFER
└─ element[1]: foo
```

▼
추출

RawRegistration	
action	ACTION_DATA_TRANSFER
callback	foo
site	table 초기화 항목

두 값이 같은 초기화 항목의 자식이므로 하나의 등록 쌍으로 해석한다

지정 초기화 — 같은 객체 안에서만 짝짓는다

허용 — 같은 객체

```
{  
  .action = ACTION_DATA_TRANSFER,  
  .fn      = foo  
}
```



ACTION_DATA_TRANSFER → foo

거부 — 서로 다른 항목

```
table[i].action = ACTION_DATA_TRANSFER;  
table[j].fn      = foo;
```



i == j 증명 불가 → 연결 안 함

부모가 같다고 증명되지 않으면 쌍으로 묶지 않는다

상관 대입 — 같은 수신자·같은 인덱스면 같은 슬롯

허용 — 같은 슬롯 k

```
table[k].action = ACTION_DATA_TRANSFER;  
table[k].fn      = foo;
```



ACTION_DATA_TRANSFER → foo

거부 — 다른 인덱스

```
table[k].action = ACTION_DATA_TRANSFER;  
table[m].fn      = foo;
```



UNRESOLVED · 증거 없음

등록 함수 — 이름이 아니라 저장까지 따라간다

호출부

```
register_handler(  
    ACTION_DATA_TRANSFER,  
    foo  
);
```

피호출부 — 인자를 따라간다

```
void register_handler(int action, Handler fn) {  
    table[next].action = action;    // 최종 저장  
    table[next].fn      = fn;  
}
```

인자 흐름

ACTION_DATA_TRANSFER → 파라미터 action → 저장 table[next].action
foo → 파라미터 fn → 저장 table[next].fn



ACTION_DATA_TRANSFER → foo

최종 쌍 저장을 못 찾으면 → REGISTRAR_STORE_NOT_REACHED

디스패치 — 분기 안에서 액션과 호출이 바로 붙는다

```
switch (frame->action) {  
    case ACTION_DATA_TRANSFER:  
        bar(frame->payload);  
        break;  
}
```

▼
같은 분기

case ACTION_DATA_TRANSFER



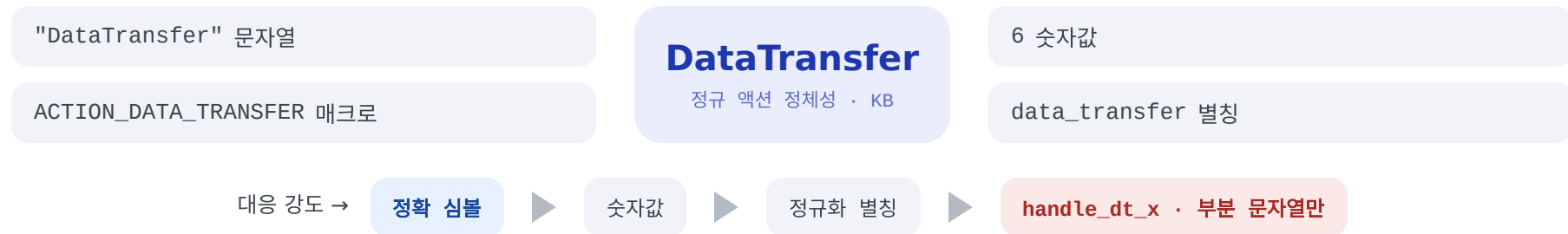
call bar()



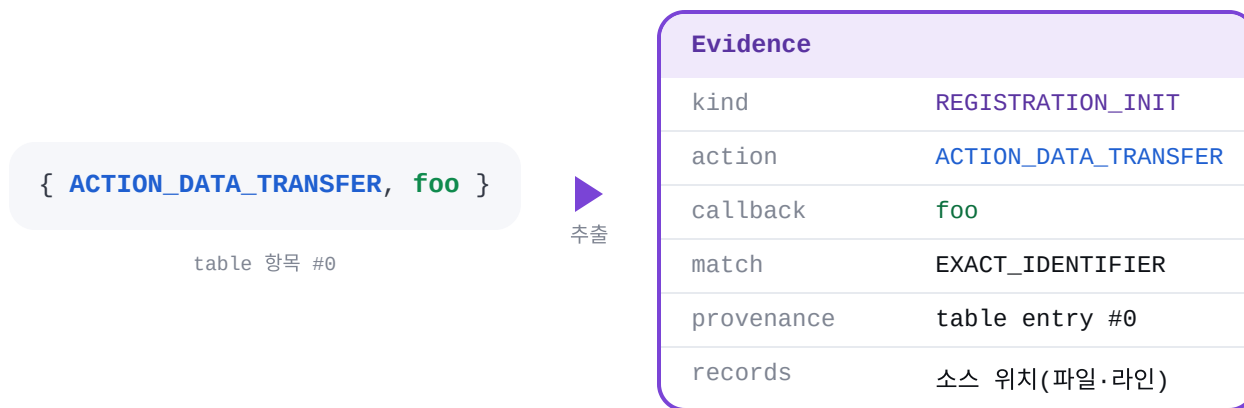
ACTION_DATA_TRANSFER → bar

동일한 case 분기 안에서 액션 선택과 호출이 직접 결합되어 있다

식별자 정규화 — 여러 표기를 하나의 정체성으로

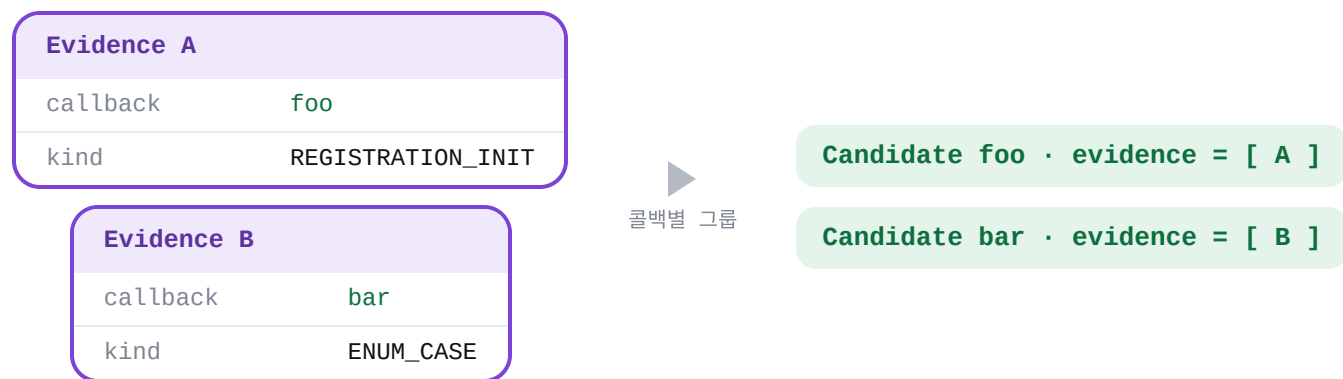


증거 객체 — 코드 조각이 각 필드가 된다



같은 색이 어디서 왔는지를 보여준다 — 각 코드 조각이 한 필드로

콜백이 같은 증거끼리 후보로 묶는다



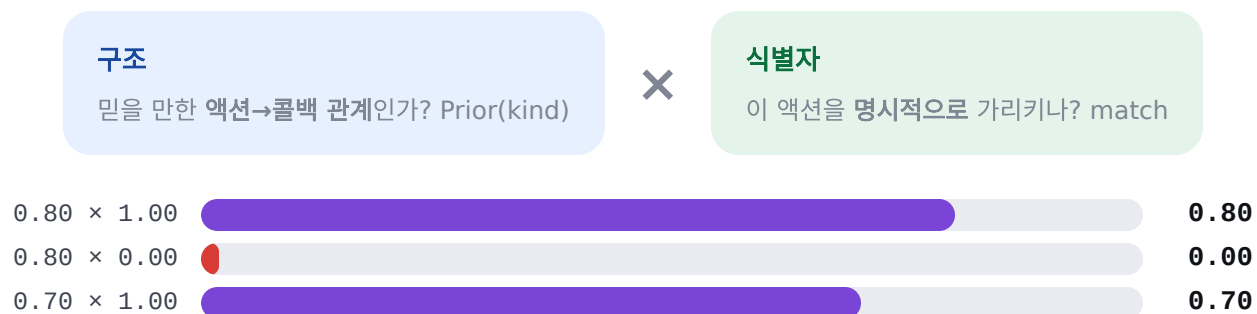
비교의 단위는 증거가 아니라 **콜백 후보** — 각 후보가 자기 증거를 들고 있다

관계를 얼마나 직접 표현하느냐가 사전확률이다

<code>strcmp(action, "DataTransfer")==0 → foo()</code>	STRING_DISPATCH		0.90
<code>case ACTION_DATA_TRANSFER: bar()</code>	ENUM_CASE		0.85
<code>{ ACTION_DATA_TRANSFER, foo } // 호출은 딴 곳</code>	REGISTRATION		0.80
<code>register(ACTION..., foo) // 피호출부 확인</code>	REGISTRAR		0.70
<code>함수 이름: handle_data_transfer // 구조 연결 없음</code>	NAME_MATCH		0.70

위로 갈수록 직접 표현, 아래로 갈수록 해석이 더 필요 · 값은 정책값이다

점수는 구조와 식별자가 모두 강해야 강해진다



둘 중 하나라도 **약하면** 전체 증거도 제한된다

같은 코드 위치에서 나온 관찰은 한 번만 센다

{ ACTION_DATA_TRANSFER, foo }

같은 출처(provenance) 하나

table-entry 관찰 · 0.80

handler-reference 관찰 · 0.80

symbol-resolution 관찰 · 0.80

셋을 다 더하면



↑ 과장

한 등록이 세 증인처럼 부풀려짐

MAX 하나로



같은 출처는 한 번만 센다 (provenance grouping)

서로 다른 곳에서 같은 핸들러를 확인하면 신뢰를 높인다

출처 A · 등록부

```
{ ACTION_DATA_TRANSFER, foo }
```

점수 0.80 · 남은 의심 0.20

출처 B · SWITCH

```
case ACTION_DATA_TRANSFER:
    foo();
```

점수 0.85 · 남은 의심 0.15

남은 의심 = $0.20 \times 0.15 = 0.03$

신뢰도 = $1 - 0.03 = 0.97$

독립된 두 출처가 확인 → 남은 의심을 곱한다 (noisy-OR)

약한 이름 셋이 구조적 증명이 될 순 없다

약한 이름 관찰 3개

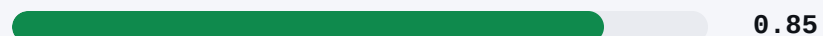
0.70 · 0.70 · 0.70



noisy-OR 그대로면 거의 확신? — 아니오

weak-only cap 적용

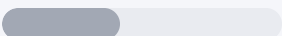
정확/해석된 식별자 증거가 하나도 없으면 →



강한 증거가 아무리 많아도 → 절대 확신엔 못 미친다 global cap 0.99

두 질문으로 확정·모호·미해결을 가른다

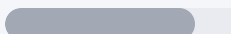
질문 1 · 1위가 충분히 강한가? **floor 0.50**

top  **0.42**

$0.42 < 0.50 \rightarrow \text{UNRESOLVED}$

질문 2 · 2위보다 뚜렷이 앞서나? **margin 0.15**

top  **0.66**

2nd  **0.56**

차이 $0.10 < 0.15 \rightarrow \text{AMBIGUOUS}$

둘 다 통과

top  **0.72**

2nd  **0.56**

차이 $0.16 \geq 0.15 \rightarrow \text{RESOLVED}$

사례 — foo vs bar 를 숫자로 실행한다

foo · REGISTRATION_INIT

prior 0.80 × match HEURISTIC_SUBSTRING 0.70



bar · ENUM_CASE

prior 0.85 × match NORMALIZED_NAME 0.85



floor · $0.72 \geq 0.50$ ✓

margin · $0.72 - 0.56 = 0.16 \geq 0.15$ ✓

RESOLVED → bar

디스패치(bar)의 식별자 대응이 더 강해 확정 — **종류가 아니라 대응 강도**의 차이

입력을 바꾸면 결과가 이렇게 달라진다

A · switch 제거

foo 0.56 단독, 경쟁 없음

RESOLVED → foo

B · 점수가 근접

bar 0.66 · foo 0.56 (차이 0.10)

AMBIGUOUS

C · 약한 증거만

결합이 0.85를 넘어도

weak-only cap → 0.85

D · 구조적 쌍 없음

후보 자체가 없음

UNRESOLVED + 사유

판정의 각 요소가 출력 JSON의 필드가 된다

RESOLVED → bar

판정 결과
1위 후보
결합 점수
모든 대안
소스 레코드



직렬화

handler_resolutions

status	← 판정 결과
chosen	← 1위 후보
confidence	← 결합 점수
candidates	← 모든 대안
evidence	← 소스 레코드
unresolved	← 실패 사유

확정된 핸들러가 오염 분석의 출발점이 된다

확정된 핸들러 · BAR

```
int bar(Payload *p) {
  int limit = p->chargingLimit;
  set_current_limit(limit);
}
```

프로토콜 필드 `chargingLimit`

지역 변수 `limit`

싱크 `set_current_limit`

핸들러가 미해결이면

미해결

신뢰할 소스 루트 없음

이 액션의 하위 분석 중단

핸들러가 없으면 오염의 출발점이 없어 분석이 멈춘다

정적으로 슬롯을 증명할 수 있는가가 경계다

지원 — 같은 슬롯

```
table[k].action = ACTION;
table[k].fn      = foo;
```

같은 수신자 + 같은 슬롯 → 쌍 복원 가능

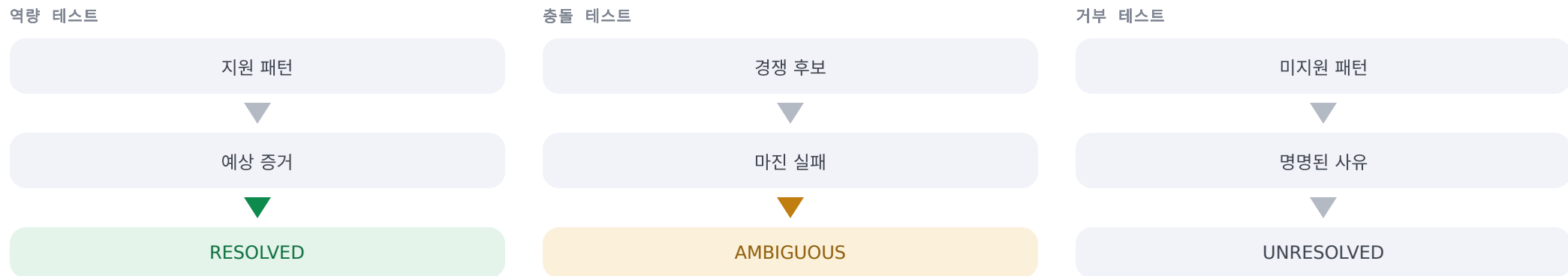
미지원 — 런타임 탐색

```
for (...) {
  if (table[i].action == action)
    table[i].fn = foo;
}
```

런타임 탐색 + 심볼릭 인덱스 → 루프·값 추론 필요 → **미해결 · 사유 명명**

전체 지원/미지원 목록은 **부록**으로 — 여기선 경계의 **이유**를 그림으로

테스트는 ‘구현 동작’을 고정하지, ‘현실 정확도’를 증명하진 않는다



고정 픽스처 테스트 = 구현 동작 증명 · 임의 펌웨어의 현실 정확도는 별개

부록 — 상수 · 사유 · 결합 방식 비교

사전확률 · 매치 배수 · 임계값

```
STRING_DISPATCH 0.90 · ENUM_CASE 0.85 REGISTRATION_* 0.80 · REGISTRAR
0.70 · NAME 0.70/0.65 EXACT 1.00 · RESOLVED_VALUE 0.95 · NORMALIZED
0.85 SUBSTRING 0.70 · NONE 0.00 floor 0.50 · margin 0.15 · weak-only
0.85 · global 0.99 · depth 2
```

미해결 사유 · 결합 방식 (0.80·0.49)

```
NO_EVIDENCE · LOW_CONFIDENCE · EXTERNAL_DEFINITION
REGISTRAR_STORE_NOT_REACHED · REGISTRAR_SEARCH_THEN_WRITE
MISSING_POINTSTO · DYNAMIC_ACTION_ID · REGISTRATION_OUT_OF_TU — MAX
0.80 · SUM 1.29 · AVG 0.645 · NOISY-OR 0.898
```

본문은 계약과 변환의 이해에 집중 · 수치·전체 분류는 여기에만